

PYTHON PROGRAMMING

L A N G U A G E

➤ Start Slide

INTRODUCTION

1. What is Python?

Definition:

Python is a high-level, interpreted, general-purpose programming language known for its readability and simplicity.

Key Characteristics:

Interpreted (not compiled)

Dynamically typed

Object-oriented

Multi-paradigm (supports procedural, OOP, functional)

2. History of Python

Created by Guido van Rossum

First released in 1991

Named after Monty Python's Flying Circus (not the snake!)

Open-source from the beginning

Major versions:

Python 2.x (discontinued in 2020)

Python 3.x (actively maintained)

FEATURES OF PYTHON

Feature	Description
Easy to Learn	Clean, English-like syntax
Interpreted Language	No need to compile
Dynamically Typed	No need to declare data types
Platform Independent	Write once, run anywhere
Vast Libraries	Includes NumPy, Pandas, Django, etc.
Community Support	Strong global community
Versatile	Can be used for web, AI, data science, etc

APPLICATION OF PYTHON

- Web Development → Flask, Django
- Data Science → Pandas, NumPy, Scikit-learn
- AI & ML → TensorFlow, PyTorch
- Automation → Scripting, Web scraping
- Game Development → Pygame
- Desktop GUI → Tkinter, PyQt
- Internet of Things (IoT)
- Blockchain, Cybersecurity

INSTALLING PYTHON

Step-by-Step (Windows Example):

- Visit <https://www.python.org>
- Download the latest stable release (Python 3.x)
- During installation, check  “Add Python to PATH”
- Verify with command:
 - `python --version`

VARIABLE

A variable is a named location in memory used to store data. In Python, variables are created when you assign a value to them.

```
>> name = "Antick"
```

```
>> age = 30
```

Rules:

- Must start with a letter or underscore _
- Can contain letters, digits, and underscores
- Case-sensitive (Age $\neq$ age)
- Cannot use reserved keywords (like if, for, etc.)

DATA TYPES

Type	Example
int	x = 10
float	pi = 3.14
str	name = "Python"
bool	is_valid = True
complex	z = 3 + 4j

DIFFERENCE OF f-SRTING & PRINT WITH COMMAS

Feature	f-string	print with commas
Readability	High	Moderate
String formatting	Precise and customizable	Limited control
Concatenation	Automatic	Done by separating with commas
Performance	Slightly faster	Slightly slower (internally builds string)
Expression support	Yes ({a+1}, {len(name)} etc.)	No

ARITHMETIC OPERATORS

Operator	Name	Example	Result
+	Addition	10 + 5	15
-	Subtraction	10 − 5	5
*	Multiplication	10 * 5	50
/	Division	10 ⁄ 5	2
//	Floor Division	10 // 3	3
%	Modulus	10 % 3	1
**	Exponentiation	2 ** 3	8

ASSIGNMENT OPERATORS

Operator	Example	Equivalent To
<code>'='</code>	<code>x = 5</code>	Assign 5 to x
<code>+=</code>	<code>x += 3</code>	<code>x = x + 3</code>
<code>-=</code>	<code>x -= 3</code>	<code>x = x - 3</code>
<code>*=</code>	<code>x *= 3</code>	<code>x = x * 3</code>
<code>/=</code>	<code>x /= 3</code>	<code>x = x / 3</code>
<code>//=</code>	<code>x //= 3</code>	<code>x = x // 3</code>
<code>%=</code>	<code>x %= 3</code>	<code>x = x % 3</code>
<code>**=</code>	<code>x **= 3</code>	<code>x = x ** 3</code>

COMPARISON OPERATORS

Operator	Description	Example	Output
'=='	Equal to	5 == 5	TRUE
!='	Not equal to	5 != 3	TRUE
>	Greater than	5 > 3	TRUE
<	Less than	5 < 3	FALSE
>=	Greater or equal	5 >= 5	TRUE
<=	Less or equal	5 <= 4	FALSE

LOGICAL OPERATORS

Operator	Description	Example	Result
and	True if both are true	<code>5 > 3 and 5 < 10</code>	TRUE
or	True if at least one is true	<code>3 > 10 or 3 < 5</code>	TRUE
not	Inverts the result	<code>not(8 > 5)</code>	FALSE

IDENTITY OPERATORS

Operator	Description	Example	Output
is	Returns True if both point to same object	x is y	True/False
is not	Returns True if not same object	x is not y	True/False

MEMBERSHIP OPERATORS

Operator	Description	Example	Output
in	True if value found	'a' in 'apple'	TRUE
not in	True if not found	'z' not in 'apple'	TRUE

CONTROL STATEMENTS

Conditional Statements

- if statement
- if-else statement
- if-elif-else ladder

Looping Statements

- while loop
- for loop

Range Function

- range(start, stop, step)

Loop Control Statements

- break
- continue
- pass

Else with Loop

- else with for loop
- else with while loop

TOKEN IN PYTHON

A "token" in Python, at its core, is the smallest meaningful unit in a program. When the Python interpreter reads your code, it doesn't process it character by character. Instead, it breaks the raw stream of characters into these distinct, identifiable tokens. Think of it like breaking a sentence into words and punctuation marks – each word and punctuation mark is a token

Types of Tokens in Python

- Keywords or Reserved Words (if, else, while, for, def, class, import)
- Identifiers (my_variable, calculate_sum, MyClass, _private_data)
- Literals (10, 3.14, "hello", True, 1 + 2j)
- Operators (+, -, ==, !=, =, +=, and, or, &, |, is, in)
- Delimiters or Punctuators ((), [], { }, ,, :, .., :, @, =, ->)

WORK PROCESS OF TOKEN

The process of breaking down a stream of characters into tokens is called lexical analysis or tokenization. This is the very first phase of the Python interpreter's execution pipeline. Here's a simplified breakdown of how it works:

Example:

code->

```
print("Hello")
```

after tokenization->

1. print

2. (

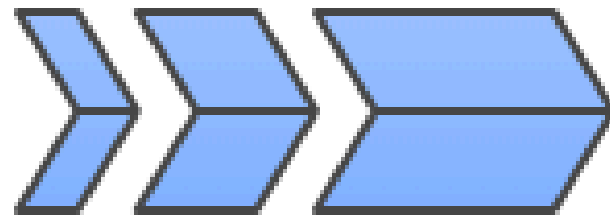
3. “

4. Hello

5. “

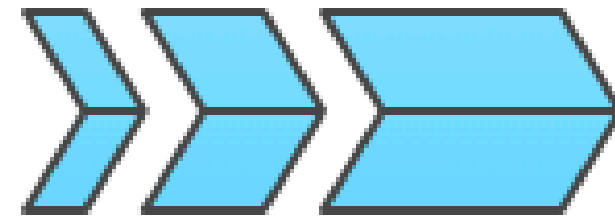
6.)

Python Interpreter Lexical Analysis Process



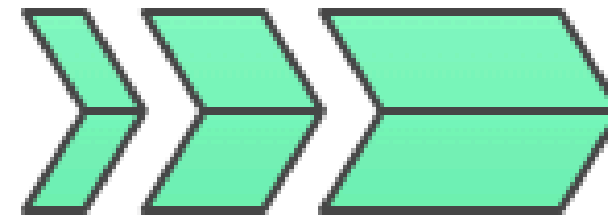
Reading Source Code

The interpreter reads the Python file character by character.



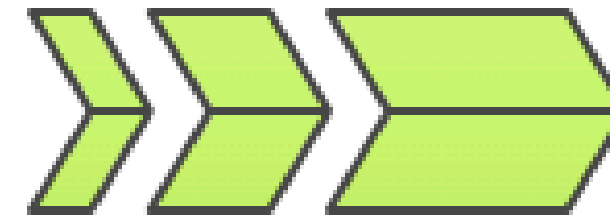
Lexer (Tokenizer)

The lexer scans and groups characters into tokens.



Rule-Based Recognition

The lexer uses rules to identify token types.



Skipping Whitespace and Comments

The lexer ignores irrelevant whitespace and comments.



Generating Token Stream

The lexer outputs a stream of tokens with type, value, and position.